

DATOS INTERÉS DE LOS TESTS U5

El DML es una parte fundamental del lenguaje SQL que se utiliza para manipular los datos almacenados en una base de datos. Esto incluye operaciones como la inserción, actualización, eliminación y consulta de datos. Las operaciones más comunes en DML son:

SELECT: Recupera datos de una o más tablas.
INSERT: Inserta nuevos registros en una tabla.
UPDATE: Modifica registros existentes en una tabla.
DELETE: Elimina registros de una tabla.

Aquí tienes una lista de los operadores más habituales en SQL, organizados por orden de precedencia (de mayor a menor). Este orden determina cómo se evalúan las expresiones cuando no hay paréntesis que alteren la prioridad:

1. Operadores aritméticos

Estos operadores tienen la mayor precedencia y se evalúan primero.

* (Multiplicación)
/ (División)
% (Módulo o residuo, si está soportado)
+ (Suma)
- (Resta)

3. Operadores de comparación

Estos operadores se utilizan para comparar valores y devolver un resultado booleano (TRUE, FALSE o NULL).

= (Igualdad)
< (Menor que)
> (Mayor que)
<= (Menor o igual que)
>= (Mayor o igual que)
<> o != (Diferente de)
IS NULL / IS NOT NULL (Verifica si un valor es nulo o no)
BETWEEN (Verifica si un valor está dentro de un rango)
IN (Verifica si un valor está en una lista)
LIKE / NOT LIKE (Coincidencia de patrones)
EXISTS (Verifica si existe al menos una fila en una subconsulta)

4. Operadores lógicos

Estos operadores combinan condiciones booleanas. Tienen la menor precedencia.

NOT (Negación lógica)
AND (Conjunción lógica)
OR (Disyunción lógica)

Ejemplo práctico de precedencia

Supongamos la siguiente expresión en SQL:

```
SELECT *
FROM tabla
WHERE 5 + 3 * 2 > 10 AND 4 = 4 OR 6 < 3;
El orden de evaluación sería:
Multiplicación (*): 3 * 2 = 6.
Suma (+): 5 + 6 = 11.
Comparación (>): 11 > 10 → TRUE.
Comparación (=): 4 = 4 → TRUE.
Comparación (<): 6 < 3 → FALSE.
Operador lógico AND: TRUE AND TRUE → TRUE.
Operador lógico OR: TRUE OR FALSE → TRUE.
```

NULL = NULL → Devuelve NULL (no es verdadero ni falso).

NULL != NULL → Devuelve NULL.

valor = NULL → Devuelve NULL, independientemente del valor.

valor != NULL → También devuelve NULL.

La operación **UNION ALL** se utiliza para combinar los resultados de dos consultas SELECT manteniendo **todos los registros**, incluidos los duplicados. A diferencia de UNION, que elimina duplicados automáticamente, UNION ALL no realiza esta eliminación y, por lo tanto, es más eficiente en términos de rendimiento cuando no es necesario eliminar duplicados.

¿Cuál es la diferencia entre IN e = en una subconsulta?

El número de valores con los que se compara.

La principal diferencia entre IN y = en una subconsulta radica en el número de valores con los que se puede comparar:

Operador =:

Se utiliza cuando la subconsulta devuelve un único valor.

Si la subconsulta devuelve más de un valor, se generará un error (por ejemplo, ORA-01427: single-row subquery returns more than one row en Oracle).

Operador IN:

Se utiliza cuando la subconsulta devuelve uno o más valores.

Permite comparar un valor con un conjunto de valores devueltos por la subconsulta.

La función COUNT(columna) cuenta el número de registros en una tabla donde los valores de la columna especificada no son NULL. Es decir, ignora las filas donde la columna tiene un valor NULL.

Comportamiento de las funciones COUNT:

COUNT(*):

Cuenta todas las filas de la tabla, incluidas aquellas que tienen valores NULL en cualquier columna.

No discrimina entre valores NULL o no NULL.

COUNT(columna):

Cuenta solo las filas donde la columna especificada tiene un valor distinto de NULL.

Ignora las filas donde la columna tiene un valor NULL.

COUNT(NOT NULL *) o COUNT(NULL *):

Estas sintaxis no son válidas en SQL y generarán un error.

Un **CROSS JOIN** es un tipo de combinación en SQL que devuelve el **producto cartesiano** de dos tablas. Esto significa que cada fila de la primera tabla se combina con **todas las filas** de la segunda tabla, generando todas las combinaciones posibles entre las filas de ambas tablas.

Comportamiento:

Si la primera tabla tiene m filas y la segunda tabla tiene n filas, el resultado tendrá m * n filas.

No se requiere ni se permite una condición de combinación (ON o USING) en un CROSS JOIN.

Si en una consulta SELECT aparecen **columnas que no están incluidas en la cláusula GROUP BY** y no se aplican funciones de agregación (como SUM(), COUNT(), AVG(), etc.), la mayoría de los sistemas de bases de datos generarán un **error**. Esto ocurre porque las columnas no agrupadas no tienen un valor único por grupo, y el motor de la base de datos no sabe cómo combinar esos valores.

Un **OUTER JOIN** es útil cuando deseas **incluir filas de una o ambas tablas que no tienen coincidencias en la otra tabla**. A diferencia de un INNER JOIN, que solo devuelve las filas que tienen coincidencias en ambas tablas, un OUTER JOIN permite conservar todas las filas de una o ambas tablas, incluso si no hay coincidencias.

Para obtener **resúmenes de datos** en SQL, se utilizan **funciones de agregación** junto con la cláusula GROUP BY (opcionalmente) para agrupar los datos y realizar cálculos sobre conjuntos de filas. Estas consultas son útiles para generar informes estadísticos, totales, promedios, conteos, etc.

Elementos clave para obtener resúmenes de datos:

1. Funciones de agregación:

- Son funciones que realizan cálculos sobre un conjunto de valores y devuelven un único resultado. Las más comunes son:
 - COUNT(): Cuenta el número de filas o valores no nulos.
 - SUM(): Calcula la suma de los valores.
 - AVG(): Calcula el promedio de los valores.
 - MIN(): Encuentra el valor mínimo.
 - MAX(): Encuentra el valor máximo.

2. Cláusula GROUP BY:

- Agrupa las filas que tienen los mismos valores en una o más columnas y aplica las funciones de agregación a cada grupo.
- Es útil para generar subtotales o resúmenes por categorías.
- Cuando usas la cláusula GROUP BY, SQL agrupa los resultados basándose en las columnas especificadas. Sin embargo, hay una regla importante:
- **Todas las columnas seleccionadas (SELECT) que no estén dentro de una función de agregación deben incluirse en el GROUP BY.**

•

3. Cláusula HAVING:

- Filtra los grupos creados por GROUP BY basándose en condiciones específicas.

Comportamiento de CEIL(n):

Si n es un número entero:

CEIL(n) devuelve el mismo valor, ya que no hay necesidad de redondear.

Si n es un número decimal:

CEIL(n) redondea hacia arriba al siguiente entero, incluso si el decimal es muy pequeño (por ejemplo, 4.0001 se convierte en 5).

Si n es negativo:

CEIL(n) sigue redondeando hacia arriba, lo que significa que se acerca a cero. Por ejemplo, CEIL(-3.7) devuelve -3.

Cuando utilizas una función de agregación (como COUNT(), SUM(), AVG(), MIN(), MAX()) en una consulta sin incluir la cláusula GROUP BY, la función de agregación se aplica a todos los registros de la tabla como un único grupo. Esto significa que el resultado será un único valor para toda la tabla.

La función **COUNT()** es una función de agregación que se puede aplicar a columnas de cualquier tipo, incluidas las columnas de tipo **texto** (strings). COUNT() cuenta el número de filas en una tabla o columna, dependiendo de cómo se utilice.

Un INNER JOIN permite unir dos tablas sin que sea necesario definir explícitamente claves primarias o foráneas. Solo necesitas especificar una condición de unión (usando la cláusula ON) que relacione las columnas de ambas tablas. Esto hace que el INNER JOIN sea flexible y no dependa de la existencia de restricciones de integridad referencial (como claves primarias o foráneas).

En una consulta SQL que incluye la cláusula GROUP BY, todas las columnas en la cláusula SELECT deben cumplir con uno de los siguientes requisitos:

Estar incluidas en la cláusula GROUP BY:

Si una columna aparece en el SELECT, debe estar incluida en la cláusula GROUP BY para que se pueda mostrar directamente.

Usar funciones de agregación:

Si una columna no está incluida en la cláusula GROUP BY, entonces debe ser utilizada dentro de una función de agregación (como SUM(), COUNT(), AVG(), MIN(), MAX(), etc.). Esto permite realizar cálculos sobre los grupos formados por GROUP BY.

El operador **UNION** en SQL se utiliza para combinar los resultados de dos o más consultas SELECT en un solo conjunto de resultados. Esto es especialmente útil en consultas multitabla cuando necesitas integrar datos de diferentes tablas que tienen estructuras similares pero no están directamente relacionadas mediante claves foráneas o relaciones.

El operador UNION en SQL se utiliza para combinar los resultados de dos o más consultas SELECT en un solo conjunto de resultados. Sin embargo, cuando se usa UNION sin la cláusula ALL, se eliminan automáticamente las filas duplicadas del resultado final.

Operador ANY:

- Compara un valor con **cualquiera** de los valores devueltos por la subconsulta.
- La comparación será **verdadera** si al menos uno de los valores de la subconsulta cumple con la condición.
- Compara un valor con todos los valores devueltos por la subconsulta.
- La comparación será verdadera solo si el valor cumple la condición respecto a todos los valores de la subconsulta.

En SQL, cuando se utiliza la palabra clave **JOIN** sin especificar explícitamente si es un INNER JOIN, LEFT JOIN, RIGHT JOIN o FULL OUTER JOIN, el sistema asume que se trata de un **INNER JOIN**. Esto significa que solo se devolverán las filas que tengan coincidencias en ambas tablas involucradas en el JOIN.

Respuesta: La principal ventaja de JOIN USING respecto a JOIN ON es que simplifica la sintaxis cuando las columnas utilizadas para la unión tienen el mismo nombre en ambas tablas, lo que resulta en una consulta más legible y concisa.

El NATURAL JOIN es una forma simplificada de realizar un INNER JOIN entre dos tablas cuando las columnas de unión tienen el mismo nombre en ambas tablas. A diferencia de JOIN ON o JOIN USING, no es necesario especificar las columnas de unión porque el sistema las detecta automáticamente.

El FULL OUTER JOIN (o simplemente FULL JOIN) es un tipo de combinación en SQL que devuelve todas las filas de ambas tablas, incluso si no hay coincidencias entre ellas. Para las filas que no tienen coincidencias, los valores de las columnas de la tabla opuesta se rellenan con NULL.

El operador MINUS en SQL es una característica específica de Oracle que funciona de manera similar al operador EXCEPT en otros sistemas de gestión de bases de datos (SGBD). Su propósito es devolver las filas que están presentes en el resultado del primer SELECT pero no en el resultado del segundo SELECT.

Cuando se utiliza un **CROSS JOIN** en SQL sin una condición WHERE, el resultado es un **producto cartesiano** de las dos tablas involucradas. Esto significa que cada fila de la primera tabla se combina con **todas** las filas de la segunda tabla, generando todas las combinaciones posibles entre ellas.